Final Competition Proposal Pitch

Donghao Hong, Junjie Kang, Vasilisa Ignatova

Competition Goal + Our Strategy

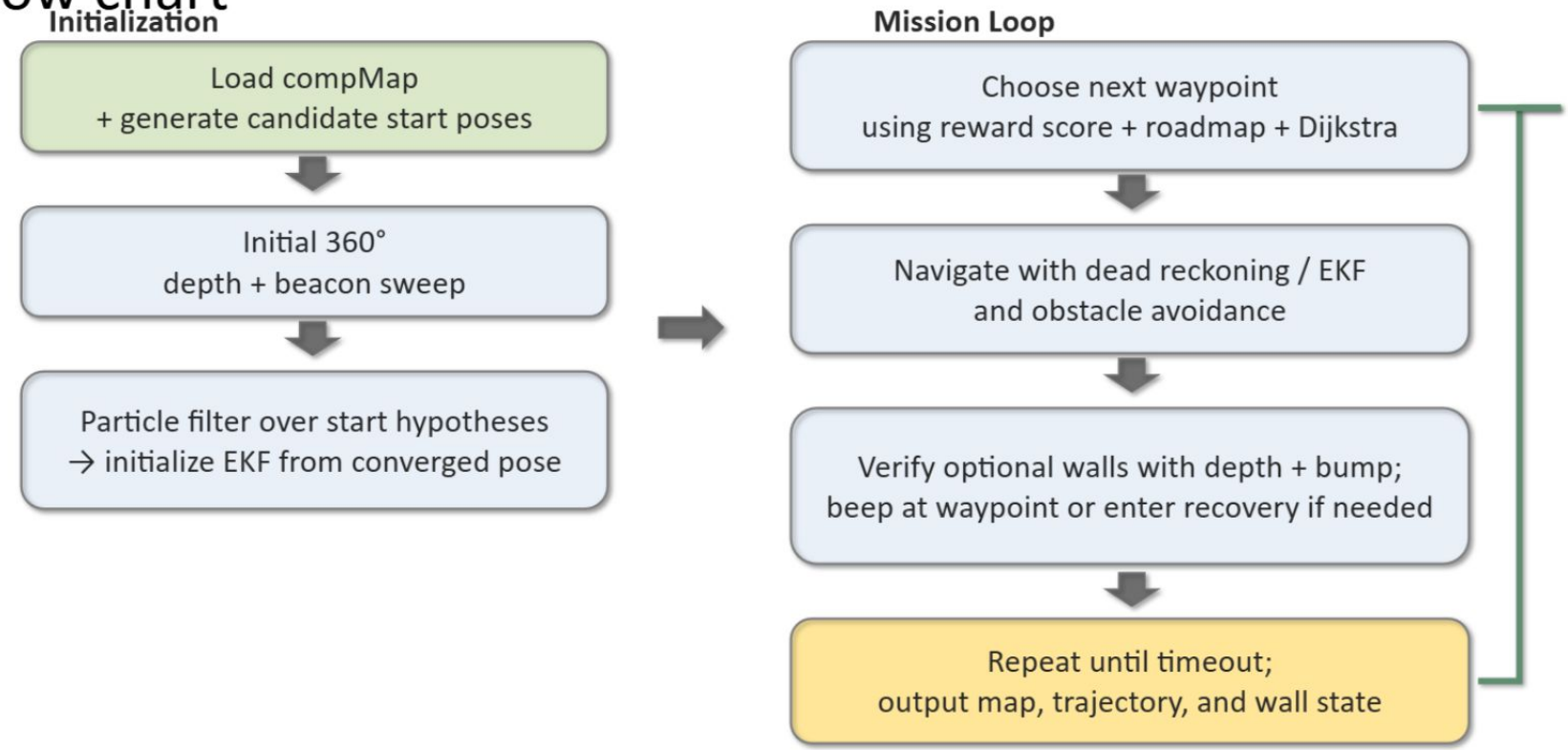
Competition Goals

- Initialize robot position without overhead localization
- Localize during movement
- Detect optional walls
- Visit as many waypoints and EC waypoints as possible

Our Strategy

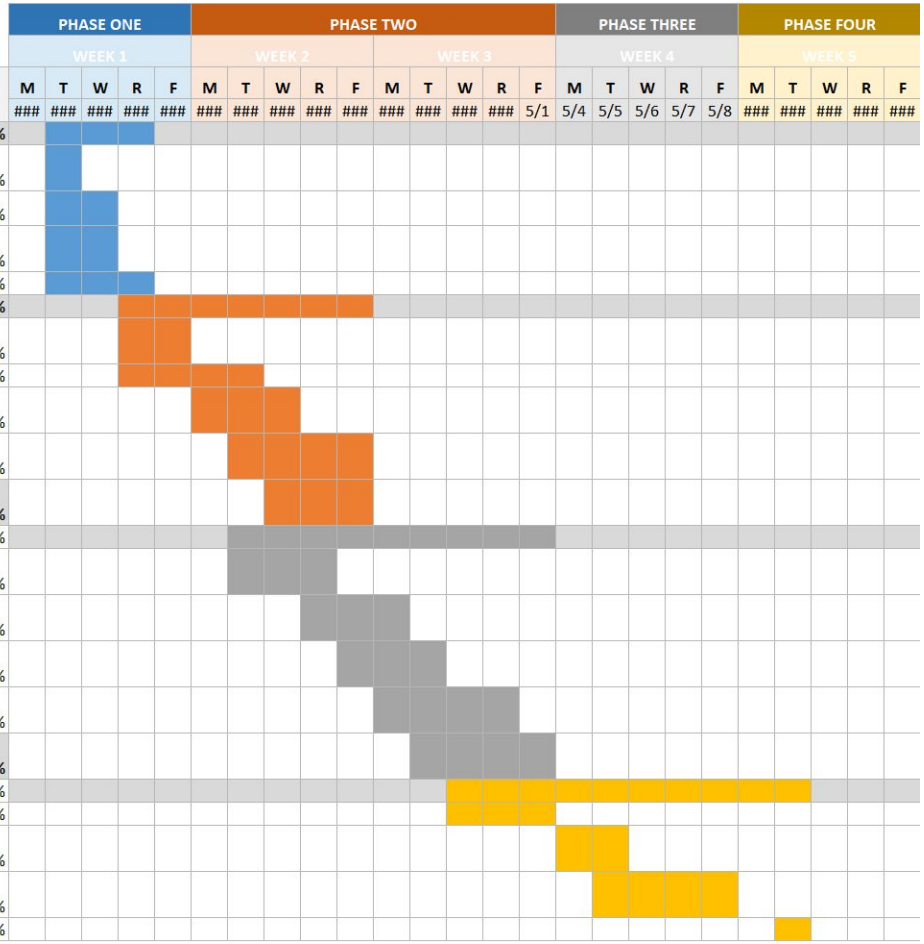
- Phase 1: reliable initialization and localization
- Phase 2: safe and score-aware navigation
- Phase 3: wall verification and map update
- Recovery behavior after bump or localization failure

Flow chart


Initialization
Load compMap
+ generate candidate start poses
Initial 360°
depth + beacon sweep
Particle filter over start hypotheses
→ initialize EKF from converged pose
Mission Loop
Choose next waypoint
using reward score + roadmap + Dijkstra
Navigate with dead reckoning / EKF
and obstacle avoidance
Verify optional walls with depth + bump;
beep at waypoint or enter recovery if needed
Repeat until timeout;
output map, trajectory, and wall state

Milestones

WBS NUMBER	TASK TITLE	TASK OWNER	START DATE	DUE DATE	DURATION	PCT OF TASK COMPLETE
1	**Design Pitch and Planning**		**4/14/26**	**4/16/26**	**2**	**100%**
1.1	Review competition requirements and scoring		4/14/26	4/14/26	0	100%
1.2	Finalize strategy, assumptions, and algorithm choices		4/14/26	4/15/26	1	100%
1.3	Update flow chart, gantt chart, and slides		4/14/26	4/15/26	1	100%
1.4	Design pitch presentation window		4/14/26	4/16/26	2	100%
2	**Initialization and Tracking Core**		**4/16/26**	**4/24/26**	**8**	**0%**
2.1	Implement initial 360° depth + beacon sweep		4/16/26	4/17/26	1	0%
2.2	Run particle filter over start hypotheses		4/16/26	4/21/26	5	0%
2.3	Initialize EKF from converged particle-filter pose		4/20/26	4/22/26	2	0%
2.4	EKF tracking with dead reckoning + depth / beacon updates		4/21/26	4/24/26	3	0%
2.5	**Beacon logging and pose-consistency checks**		**4/22/26**	**4/24/26**	**2**	**0%**
3	Navigation, Wall Logic, and Integration		4/21/26	5/1/26	10	0%
3.1	Build roadmap for safe and reachable targets		4/21/26	4/23/26	2	0%
3.2	Dijkstra-based waypoint planning with reward / distance score		4/23/26	4/27/26	4	0%
3.3	Optional wall verification using depth + bump sensing		4/24/26	4/28/26	4	0%
3.4	Recovery mode + blind-navigation fallback		4/27/26	4/30/26	3	0%
3.5	**Reuse existing functions and integrate into finalCompetition.m**		**4/28/26**	**5/1/26**	**3**	**0%**
4	Testing, Competition, and Reporting		4/29/26	5/12/26	13	0%
4.1	End-to-end simulator and lab testing		4/29/26	5/2/26	3	0%
4.2	Final tuning, code freeze, and Canvas upload		5/4/26	5/5/26	1	0%
4.3	Competition run + post-process trajectory / wall results		5/5/26	5/8/26	3	0%
4.4	Individual final report submission		5/12/26	5/12/26	0	0%



Position Initialization

Goal: identify starting waypoint and heading

- Perform and initial map sweep (record any beacons seen)
- Run the particle filter during the sweep with 100 particles for the 30seconds to a first minute or until the likely location estimates converge
- Take the converged or last pose estimate and start EKF depth estimation for the next stage of the competition

Advantages:

- Particle filter can keep track of several location hypotheses -> good for the lost robot problem

Disadvantages:

- Computationally heavy
- Likely estimate results may never converge due to an incomplete map of the world

Position Tracking

- EKF with beacon and depth sensor readings
- Dead Reckoning estimates

Advantages:

- EKF is less computationally demanding than the particle filter
- It combines prediction and sensor correction for more stable pose estimate

Disadvantages:

- Need a good initial estimate for EKF
- Odometry integration accumulates error over time

Wall Detection

- Once the robot is localized and traveling continuously check if you are close to the location of a wall that needs to be verified
- Perform a verification check using both a bump sensor and a depth sensor

Anticipated Problems:

- False verification result due incorrect robot orientation
- Bump and depth sensor may not come to a consensus regarding the existence of a wall
  - Will need a strategy for making a decision

Navigation

- Choose next waypoint based on Euclidian distance and expected reward
- Prioritize safe and reachable targets (build a roadmap)
- Replan when new wall information becomes available
- Prefer efficient paths under the time limit (use Dijkstra's algorithm)
- If roadmap fails, switch to blind navigation with depth sensors for obstacle avoidance

Recovery mode

- Triggered by bump detection, high pose uncertainty, or navigation failure
- Stop and back away form the obstacle
- Reorient and collect new depth / beacon observations( )
- Re-localize using updated sensor data
- Replan and continue toward the waypoint

Integration Plan

Existing Functions Reused:

- Visitwaypoints
- EKF Depth + Particle Filter
- Roadmap functions

New Functions Needed:

- Function for choosing next waypoint based on proximity
- Obstacle avoidance/recovery routines